

NOVA'S FULLSTACK SYLLABUS

PRE Jun 13–17	01 Backend Jun 18–Jul 4	02 Database Jul 5–15	03 React Jul 16–Aug 1	04 AI Aug 2–8	05 Deploy Aug 9–15	PROJECT
------------------	----------------------------	-------------------------	--------------------------	------------------	-----------------------	---------

PRE Non-Negotiable Foundations

5 Days · Jun 13–17

JavaScript Core

- let/const, arrow & normal functions, scope
- Objects, Arrays, destructuring, spread/rest
- map, filter, reduce — know cold
- Promises & async/await — how they work
- try/catch error handling

Internet & Auth Concepts

- HTTP, REST, JSON — full browser→server flow
- Status codes: 200, 201, 400, 401, 404, 500
- SQL vs NoSQL — concept only
- Authentication vs Authorization
- JWT tokens — concept, not code yet

✓ Explain async/await, REST, JWT, SQL vs NoSQL out loud. No looking. Confident? → Phase 1.

01 Backend Engineering — Node.js + Express + Auth

2.5 Weeks · Jun 18–Jul 4

Node.js + Express

- Event loop — understand in real code
- Modules: require / import-export
- .env variables — never hardcode secrets
- MVC structure: Routes→Controllers→Services
- Middleware: custom + built-in (cors, json)
- CORS — what it is, how to fix it
- Centralized error handling middleware
- Rate limiting (express-rate-limit)
- Multer — file uploads
- Postman — test every route

Auth System

- User schema design
- bcrypt — password hashing
- JWT generation + verification middleware
- Access token vs refresh token
- Role-based access (RBAC) — basic
- Protected route middleware
- Logout strategy
- helmet.js + input sanitization basics

✓ Build full Auth API: register, login, protected route, RBAC, file upload. Every route tested in Postman. No tutorial.

02 Database Engineering — MongoDB + Mongoose

1.5 Weeks · Jul 5–15

MongoDB (Do This)

- Schema design: embed vs reference
- Mongoose models, validators, defaults
- CRUD: create, find, update, delete
- Mongoose populate (join-like queries)
- Indexing — single & compound
- Pagination & filtering patterns
- MongoDB Atlas — cloud setup

SQL Awareness (Light Touch)

- SELECT, JOIN, WHERE, GROUP BY — read only
- When SQL vs NoSQL — decision thinking
- Indexes in SQL — concept only
- No deep SQL needed at this stage

✓ Design project schema. Write 3 queries with populate + pagination. Explain schema choices aloud.

03 Frontend — React Core + Full API Integration

2.5 Weeks · Jul 16–Aug 1

React Core

- JSX & component thinking
- Props, state, lifting state up
- useState — master this first
- useEffect — fetching, cleanup
- useRef — DOM access
- useContext — simple global state
- Custom hooks — reusable logic
- React Router — dynamic + protected routes

Integration

- Axios/Fetch — API calls from React
- Loading + error states (always both)
- JWT in localStorage + auth guard
- File upload UI — progress + preview
- Form handling + validation
- Feature-based folder structure

AI Integration (Awareness)

- What is an API key — never on frontend
- Gemini API (free) — easy setup
- Flow: Button → Backend → Gemini → UI
- Basic prompt engineering
- Error handling + fallback
- One feature: Summarize note with AI

SKIP FOR NOW: RAG, LangChain, Vector DB — not now

Deployment + Docker

- Vercel — React frontend (free)
- Render/Railway — Node backend (free)
- MongoDB Atlas — production DB
- ENV variables in production
- Fix CORS for production URLs
- Dockerfile for Node backend
- docker build + docker run
- Docker Compose — Node + Mongo local

✓ AI feature working. Frontend on Vercel. Backend on Render. Docker builds locally. Share the URL.

